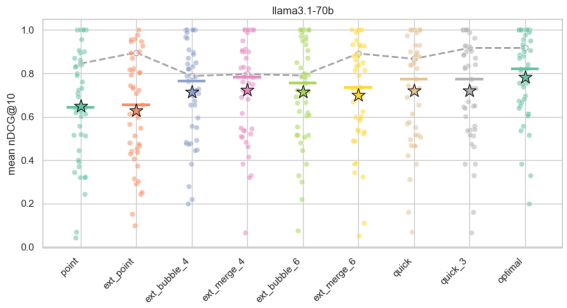

Figure 2: Accuracy distribution across algorithms for the LLaMA 3.1 70B model. Each dot represents a query; star markers denote the mean accuracy, and bars indicate the median.

sort and external merge sort perform similarly on Llama, but the former significantly outperforms the latter on GPT 4.1.

DL19 Passage Ranking. The second row of Figure 1 reports the evaluation of different algorithms in DL19. The x-axis is the dollar cost and the y-axis is the mean *nDCG*@10 across 43 queries (higher is better). For passage ranking tasks, comparison-based sorting algorithms excel. Our proposed semantic external merge sort and quick sort with majority voting consistently achieve the best mean *nDCG*@10. On the other hand, the value based approach performs worse for passage ranking. When the task is not centered on deriving factual information, value-based approaches are less effective, as they lack the comparative reasoning needed to capture subtle relevance signals. This phenomenon has also been observed in many prior works [37, 47].

Trade-off between Sorting Accuracy and Computation

Figure 1 illustrates the relationship between the accuracy of the sorting (Kendall's tau or*nDCG*@10) and the price (calculated cost based on the input tokens and the output tokens consumed). In many cases, the curves follow a log-linear trend: accuracy improves rapidly with small increases in tokens, but quickly plateaus out as token usage grows. For the NBA dataset, the fitted curve excludes the pointwise and external pointwise approaches, as we suspect their strong performance stems from memorization of factual data (e.g., Wikipedia) rather than from generalizable reasoning ability. For DL19, the fitted curve includes all points. Although counterexamples to this hypothesis exist, the overall pattern suggests that more compute typically lead to improved sorting accuracy.

A universal best algorithm? It is clear that between NBA player height (factual data) ranking and DL19 (passage relevance) there is no single optimal algorithm. Next, we look into the different user queries (a total of 43 queries) contained in DL19. An example query is rank passages based on their relevance to *anthropological definition of environment*. Another example query is to rank based on relevance to *what is wifi vs bluetooth*.

Figure 2 demonstrate the accuracy distribution for each algorithm for the Llama 3.1-70b model. The star and short bar denotes the mean and the median, respectively. The x-axis is the different algorithms and the y-axis is the *nDCG*@10. The sorting accuracy can change drastically by using different algorithm. The dashed line tracks the accuracy for the query: *definition declaratory judgment*. It demonstrates a case where both pointwise sort have better ranking quality than external bubble sort, despite the fact that the mean and median accuracy of pointwise sort are generally lower than those of external bubble sort. This highlights that, even within the same passage ranking benchmark, there is still no universally optimal algorithm across all queries. In a world where we know exactly which algorithm performs best for a given instance, we could leverage these specific strengths to maximize ranking effectiveness for every user query. The rightmost *optimal* algorithm in Figure 2 illustrates this scenario. It achieves a mean *nDCG*@10 of 0.779, outperforming (+0.056) the best single algorithm, external merge sort (batch size 4), which reached a mean *nDCG*@10 of 0.723.

There is no single optimal algorithm for all LLM Order By queries. For factual data, such as NBA player heights, pointwise method achieve high accuracy with minimal computation. For reasoning tasks like DL19 (43 queries), comparison-based algorithms such as our proposed external merge sort demonstrate high average accuracy. "Zooming into individual queries, we find that the optimal strategy is highly inconsistent. Since no single algorithm is universally optimal, the best strategy is to select one dynamically based on the query, LLM model, and data characteristics.

5 OPTIMIZER DESIGN: LLM-GUIDED AND PESSIMISTIC STRATEGIES

The primary challenge in designing an efficient optimizer for selecting the best physical sorting algorithm is both estimating ranking quality without exhaustively executing all algorithms on all data points and understanding the quality without ground truth labels. As a result, we look back at classical database query optimization literature [9, 17, 67]. A recurring trend is the reliance on a subset of data samples or data summaries to inform algorithmic decision-making. As a result, we build our optimizer by first sampling a subset of the data. Then, we can run several candidate algorithms in parallel, analyze the resulting performance (e.g., monetary cost and accuracy) metrics, and determine the near-optimal configuration. Although this dynamic approach sounds promising, implementing it in practice is extremely challenging due to the potential error introduced at the sampling step and the inherent difficulty of evaluating the quality of algorithms' execution results.

5.1 Cost Estimation

Before diving into the design of our optimizer, we must first establish a methodology for monetary cost estimation. Unlike traditional database queries where the result is always 100% accurate, AI SQL queries have inherent quality and cost trade-offs. Often times, the user wants to obtain a high quality result constrained by a cost budget. As a result, our optimizer needs to select the algorithms that satisfy the user's cost constraint.

Estimating the monetary cost for each algorithm (for fixed parameters) is relatively straightforward. In Table 1, we have written the number of LLM calls for each algorithm based on the number of input data points and algorithm parameters. After the sampling step, we run several algorithms in parallel and obtain their running costs. This empirical measurement is crucial, as it provides the actual running cost to calibrate the theoretical cost models. Specifically, we can directly scale the observed cost per LLM call

across the different algorithms and input sizes to accurately project the total expenditure for the original datasets. By the law of large numbers, the higher sample ratio will lead to lower estimation error. In our experimental section, we show that a small sample size can already provide very accurate monetary cost estimation.

Example 5.1. *Assume the original dataset contains 100 data points and the user want to obtain a fully sorted list based on some criteria. The cost for running the pointwise sorting algorithm on the sample of size 20 is* \$0.2. *Then our estimation on running the pointwise sorting algorithm over the entire dataset will be* $\$0.1 \cdot \frac{100}{20} = \$0.5$, *since pointwise sort LLM calls scale linearly.*

Example 5.2. *Assume the original dataset contains 100 data points and the user want to obtain a fully sorted list based on some criteria. The cost for running the external bubble sort algorithm with batch size of four on the sample of size 20 is* \$0.15. *Then our estimation on running the external bubble sort algorithm with batch size of four over the entire dataset will be* $\$0.3 \cdot \frac{100^2/4^2}{20^2/4^2} = \$3.75$, *since external bubble sort LLM calls scale quadratically.*

5.2 World Knowledge

In Section 4, we find that for ordering based on world knowledge tasks, LLMs (e.g., Llama 3.1 and OpenAI models) can directly recall factual data. The memorization of training corpora, such as Wikipedia, is an inherent characteristic of large language models and is essential for emergent reasoning abilities [8]. Furthermore, studies indicate that such memorization is strongly correlated with model performance on factual tasks [39, 63]. Consequently, the pointwise method achieves high accuracy with minimal cost. This observation explains why many LLM-powered analytic systems employ pointwise methods for ordering operations by default [28, 62, 71]. The derived values in pointwise method act as a direct and high-confidence probe of the model's parametric memory. Motivated by this observation, the challenge for ordering based on world knowledge becomes: *Can we successfully identify whether the information needed to order the items is world knowledge already present in the LLM's training corpus?* To address this question, we follow the methodology established in the field of membership inference in LLMs, and adopt the Inquiry Prompt from [66] as shown in Prompt 4, which has achieved high membership inference accuracy and relies only on the generation API. In the evaluations, we find that the Inquiry Prompt effectively distinguishes between memorized world knowledge and unknown entities.

Prompt Block 4: Inquiry Prompt

```
You have seen the following {criteria}: "{key}" in your
     training data?
If you have seen it, return 'Yes'. Otherwise, return 'No'.
Explain if you remember or recall it.
```

We run the inquiry on a sample of the input data and if 100% of the data points are identified as members of the training corpus, the optimizer automatically uses **pointwise** ranking strategy. We use a high and strict threshold because: we can always fall-back to using the LLM-as-Judge or consensus aggregation approach described later in this section. Although the pointwise method excels in sorting based on factual knowledge, its effectiveness in general semantic ordering is fundamentally limited, as it lacks the relative comparative context inherent in comparison-based approaches. When applied to data outside the model's parametric memory, the pointwise scoring can suffer from poor calibration. By enforcing a strict membership threshold, we ensure that this rule of using the pointwise method is implemented only when the model functions as a reliable knowledge retriever.

5.3 Candidate Algorithms

The optimizer is tasked with dynamically selecting one algorithm to execute the `LLM ORDER BY` operation. The initial candidates include all value-based and comparison-based algorithms. We note that within our candidate pool, comparison-based algorithms: external bubble sort and external merge sort—introduce a functional dependency on the configured *batch size*. While numerous valid batch size configurations exist, constrained only by the model's context window, it is computationally too expensive to empirically explore every configuration. Hence, we leverage our main insight derived from Section 4: **higher computation generally leads to better accuracy**. This finding implies that to maximize the performance potential within a limited budget, we only need to explore the minimum viable batch size configuration for external comparison based algorithms, thereby ensuring the available budget is utilized toward the most promising algorithm candidate. To estimate the running price for each batch configuration, we perform a limited empirical sampling (monetary cost associated with a single LLM invocation on the number of batch size samples) and then extrapolate the total cost by scaling the observed cost per unit of work. This scaling factor is shown in Table 1.

For all algorithms, we first execute each algorithm on a small sample subset. We then project these costs to the full dataset size according to Table 1, filtering out any methods where the estimated expense violates the user-specified budget. This candidate algorithm selection phase offers flexibility and generalizability. When a new semantic sorting algorithm is proposed or discovered, integrating it into the optimizer merely requires determining its cost function (the LLM function call complexity shown in Table 1) and its minimum viable batch size (if applicable) to immediately include it in the competitive candidate pool.

The main challenge With a pool of strong candidate algorithms selected based on user-defined cost constraints, the remaining core task is to identify the best algorithm with respect to the quality of the ranking. Note that we always select a single algorithm from the candidate pool and run this algorithm over all data points once. To address the challenge of understanding the ranking quality, we introduce two distinct methodologies for measuring and evaluating the quality in the next two subsections.

5.4 LLM-as-Judge Optimizer

A common approach in evaluating complex tasks is the emergence of "LLM-as-a-Judge", where LLMs are employed as the evaluator. LLM-as-a-Judge approach has been adopted in many different domains [21]. In the context of database, the researcher has used LLM-as-a-Judge to evaluate LLM powered natural language to SQL

query translation [26, 72] and to evaluate LLM generated candidate query plans [24, 55].

Inspired by the previous success of using LLM as a Judge, we explore if we can also leverage an LLM to determine the best algorithm for a sorting task. We run all candidate algorithms on the sampled data. The resulting list of output rankings, each tagged with a unique candidate identifier, is then compiled and presented. The LLM Judge is subsequently tasked with evaluating this aggregate output and identifying the candidate identifier corresponding to the optimally sorted result for the given input. The prompt 5 shows the template used to generate the LLM Judge decision, which include the task instruction, the sampled keys, and the execution results of candidate algorithms. This chain-of-thought [69] prompt is designed in accordance with industry-standard evaluation frameworks such as TruLens [1].

Prompt Block 5: LLM-as-a-Judge Prompt

```
You are an EXPERT RANKED RESULT RATER. You are given a
     ranking criteria and a list of CANDIDATE RANKED RESULTS
Your task is to find the best ranked result which sort the
     input keys based on the ranking criteria.
keys: {keys}
Ranking criteria: {criteria}
CANDIDATE RANKED RESULTS:
{rankings}

You should think step by step about the ranking and return
     the integer identifier of the best ranking.
```

5.5 Pessimistic Optimizer

Despite the success in the "LLM-as-a-Judge" paradigm, the reliability of LLM judge's decision remains a concern. LLM is known to be inherently subjective and has the potential for hallucination [29]. Moreover, the **fundamental limitation** associated with using a single static algorithm for all ranking queries (e.g., the pointwise in BigQuery or quick sort in Lotus) is the accuracy instability: excelling on one specific query while failing significantly on others due to the diverse reasoning requirements of semantic Order By.

Therefore, to mitigate risk and establish a near-optimal solution, we introduce the heuristic pessimistic optimizer. We adapt concepts from the collective voting literature [23, 50]. Specifically, we treat the ranking order produced by all comparison-based algorithms as a set of expressed preferences among the sorted order of items based on user defined criteria. Consequently, to aggregate these diverse 'votes' into a single, robust consensus ranking, we employ the Borda Count method [14], which assigns scores based on position to determine the overall collective preference.

We calculate the **Borda Count** score for each item to derive a **gold ranking** that represents the collective preference of the comparison-based algorithms. We then treat gold ranking as the proxy source of truth and use it to evaluate the ranking quality of all candidate algorithms. The algorithm whose output exhibits the highest correlation with this derived gold ranking is formally designated the heuristic optimal choice. Although value-based approaches are not used to derive the gold rank, their quality is assessed in selecting the heuristic optimal choice. The decision to not include value-based approaches in deriving the gold ranking is that our observations (Section 4) and prior works [47, 78] have found for complex reasoning tasks (e.g., passage ranking), value-based methods significantly under-perform due to the lack of relative context and the tendency of LLMs to unstable scores. Since we have already filtered out the factual queries (where pointwise excels) using membership inference at, including uncalibrated value-based rankings in this stage would introduce detrimental noise into the consensus ranking.

6 EXPERIMENTS

We now present an evaluation of our proposed LLM-as-Judge and Heuristic optimizers (Section 5), building on top of value-based and comparison-based LLM sorting algorithms (Section 3). With systematic evaluation, we demonstrate that our proposed optimizer can achieve near-optimal accuracy through online evaluation.

6.1 Experimental Setup

Evaluation Datasets.

(1) 2020 World Population [46]: It contains 200 regions and their population. The query is to order these countries by their population. Populations are masked.
(2) TweetEval [6]: We use four specific sentiment categories from this benchmark: irony, hatefulness, positivity, and offensiveness. The query is to rank the highest 10 tweets by the intensity of the sentiment.
(3) SembenchMovie Q9 [28]: Orders the movie review for *Ant-Man and the Wasp* by positivity. We slightly modified this query by adding the **DESC LIMIT 10** clause at the end. We adopt the same pointwise prompt presented in the paper for this query.
(4) DL20 [12]: It contains 54 search queries associated with the MS MARCO passage corpus. The query is to re-order the top 100 candidate passages selected using BM25 [32] based on their relevance to the search query limit 10.

Evaluation Metrics. Similarly to the initial evaluation (Section 4), we use kendall's tau to measure the accuracy of the sorted list in world population, and use nDCG@10 for the rest. On both metrics, higher value (closer to 1.0) indicates better sorting quality.

Ranking Solutions. In this section, we compare our proposed optimizer against a comprehensive set of single-algorithm solutions. For the **value-based** category, we include both external pointwise and standard pointwise algorithms. The pointwise method serves as a key approach as it is implemented in academic prototype and utilized by BigQuery for LLM-based ordering [28, 71]. For the **comparison-based** category, we evaluate standard quick sort and external bubble sort, alongside with our proposed methods: quick sort with majority voting and external merge sort. We also note that Lotus [45] utilizes the quick sort method. The fundamental distinction between our proposed optimizer and these baselines is that: while single-algorithm solutions apply a static strategy regardless of the input, our optimizer dynamically selects the near-optimal algorithm at runtime.

Implementation Details. For all experiments in this section, we use Llama3.1-70b or Llama3.1-405b models directly accessed using Snowflake Cortext API. The temperature is set to 0 and the max